Problem PRD: Developer Productivity Dashboard

Document Owner: Navneet Shukla | Last Updated: 5-Jun-2026

Jira Epic	Link epic here
Delivery Date	TBD - to be set after PRD sign-off
Document Status	Draft
Product Lead	Navneet Shukla
Engineering Lead	TBD
Design Lead	TBD
QA Lead	TBD
Relevant Documents	Developer-Dashboard CLI repo, Claude Console export guide, Jira CSV export guide

Table of Contents

1. Overview

Background

Company_Name's engineering team is in the middle of a deliberate shift: developers who previously wrote code manually are now using Claude Code as their daily driver for completing deliverables / tasks / story points. This change is expected to increase output speed and quality - but today there is no way to measure whether that is actually happening.

Managers want to track per developer performance, AI adoption among the developer, which developers using AI (claude code) effectively and efficiently. Developers want to see their daily / weekly / monthly token usage - which claude code's dashboard already shows, but their no way to measure the true effect of shifting towards agentic coding.
Two data sources exist side by side but are never joined: Claude Console records how many tokens each developer spent and how many lines of code they produced using Claude Code. Jira records every deliverable and task each developer completed, tagged by sprint, epic, and story points. Neither system alone answers the real question - together, they do.

Why are we doing this?

We are making a company-level bet on AI-assisted development. Without a dashboard, we have no way to know:

- Whether AI usage is translating into more deliverables, faster.
- What the cost-per-deliverable is for each developer, and whether it is improving over time.
- Which developers need coaching, and which are already getting strong ROI from the tool.
  This dashboard is the measurement layer for that bet.

How do we know this is a real problem worth solving?

- The data already exists. Claude Console and Jira are both in use. The gap is joining them.
- We have already built a CLI proof-of-concept (Developer Dashboard) that merges these two data sources and produces the right metrics. The output.csv proves the data model works. What is missing is a product that makes the data visible to the people who need it.
- Managers currently have no single place to see team-wide AI adoption. They have to ask someone to run a script and share a CSV.
- Developers have no visibility into their own numbers. They do not know their token spend, their efficiency score, or how they compare to peers.
- Trends over time are invisible. The CLI overwrites its output each run. There is no history.

How does this fit into company objectives?

- ROI measurement on Claude licences: the company is spending on Claude Code seats. This dashboard is how leadership validates that investment.

- Engineering quality and velocity: linking AI usage to deliverable output and bug rates gives managers an evidence-based view of team health.
- Culture of transparency: developers seeing their own data reduces the ‘black box’ feeling and creates a self-improvement loop.

What problem is this solving?

We have shifted our engineering workflow to AI-assisted development but have no way to see whether the shift is working, who is driving it, or what it is costing per unit of output. The data to answer all of these questions exists - it just lives in two separate systems and is never visualised together.

Who are we solving for?

- Engineering managers - need a team-level view: who is using AI, who is delivering, who has a bad ratio, and how is the trend moving.
- Individual developers - need a personal view: how much have I spent this week/month, how many deliverables did I close, what is my efficiency score, and am I improving?

What are the user benefits?

- Managers: replace a manual CSV workflow with a live dashboard. Spot underperformers by flag, track team AI adoption, and report on AI ROI to leadership.
- Developers: see their own token spend vs. output in real time. Understand what their efficiency score means. Track personal improvement over time.

2. Goals

Build a web dashboard that joins Claude Console token/spend data with Jira deliverable data, per developer, filterable by week and month, so that engineering managers and developers can see AI usage and output in one place.

What success looks like

- An engineering manager opens the dashboard and immediately sees a ranked table of every developer: how many tokens they spent, how many deliverables they closed, their efficiency score, and a green/yellow/red flag - for the current week or month, selectable.
- A developer opens their profile and sees their own numbers: token spend, lines of code, deliverables closed, story points, efficiency score, and a chart showing how those metrics have moved week-over-week and month-over-month.
- The data is refreshed by uploading two CSV files (Claude Console export + Jira export) through the web UI. No command-line access is needed.

- Historical data accumulates so trends become visible across at least one quarter.

3. Success Metrics

How do we know we have solved this problem?

Metric	Definition	Target
Dashboard adoption	% of engineering team who view the dashboard at least once per reporting period	80% of team within 60 days of launch
Manual CSV sharing eliminated	Number of manual output.csv files shared via Slack or email after launch	Zero within 30 days of launch
Data freshness	Time between a Jira/Claude Console export and results visible in the dashboard	< 5 minutes after upload
Historical depth	Months of trend data stored and browsable in the UI	3+ months within 90 days of launch
Developer self-service	% of developers who have viewed their own profile page without manager prompting	60% within 60 days of launch

4. Assumptions & Dependencies

Assumptions

- Developers use the same email address (company provided) in Claude Console and in Jira. No identity mapping is required.
- The Claude Console CSV export includes: developer email, token spend (USD), and lines of code written via Claude Code, per reporting period.
- The Jira CSV export includes: assignee email, issue key, issue type (Story/Task/Bug), story points, resolved date, sprint, and epic key.
- A **'deliverable'** is defined as a Jira Story or Task in a **Done/Closed** state within the selected time window. Bug tickets are tracked separately and used for **quality scoring**, not counted as deliverables.
- The reporting week is Monday-Sunday. The reporting month is calendar month.
- For v1, data ingestion is manual: an admin downloads the CSV exports from Claude Console and Jira and uploads them through the web UI. API-based auto-ingestion is a future phase.
- The existing Python CLI pipeline (merge.py and src/ modules) is not rewritten for v1 the web app wraps and invokes it.
- The app is an internal tool. Authentication and role-based access are out of scope for v1; all users see all data.

Dependencies

- Claude Console: must be able to export a CSV of token spend and lines of code per developer per period. Exact column names and date granularity must be confirmed before build starts.
- Jira: must be able to export a CSV of closed issues with assignee, story points, resolved date, sprint, and epic. Exact export template must be confirmed.
- Existing CLI pipeline: merge.py and src/ modules must remain passing throughout the web app build. The web app calls this pipeline as a subprocess or imports it as a library.
- Backend web framework: to be decided (see Questions). Needs to handle file upload, pipeline invocation, data storage, and a REST/JSON API for the frontend.
- Database: a persistent store for historical output data per period. To be decided (see Questions).
- Frontend: a web UI framework for the dashboard, leaderboard, trend charts, and developer profiles.

5. Requirements

5.1 Data ingestion - CSV upload

#	Title	User story	Acceptance criteria	Priority	Notes
1	Upload CSV pair	As an admin, I want to upload a Claude Console CSV and a Jira CSV together through the web UI so that I do not need command-line access to refresh the dashboard.	An upload screen accepts two files: Claude Console export and Jira export. The user selects a period type (week or month) and the period value (e.g. 2026-W23 or 2026-05). On submit, the pipeline runs, results are stored, and the dashboard reflects the new data within 5 minutes.	Must have	Period picker must validate format. Reject uploads if both files are not present.
2	Upload validation & error display	As an admin, I want to see a clear error message if my CSV files are malformed so that I know what to fix before re-uploading.	If the pipeline fails (wrong columns, missing emails, parse errors), the UI shows the specific error message from the pipeline. The	Must have	Do not silently swallow pipeline errors.

#	Title	User story	Acceptance criteria	Priority	Notes
			previous period's data is unaffected.		
3	Upload history log	As an admin, I want to see a log of past uploads so that I know when data was last refreshed and whether any uploads failed.	A log shows: timestamp, period, file names, status (success/failed), and error summary if failed. Last 20 entries shown.	Must have	Persist log in the database.

5.2 Team leaderboard - manager view

#	Title	User story	Acceptance criteria	Priority	Notes
4	Team leaderboard table	As an engineering manager, I want to see all developers ranked in a single table so that I can compare AI usage and output across the team at a glance.	A table lists all developers for the selected period. Columns: developer name, tokens spent, lines of code, deliverables closed, story points delivered, AI efficiency score, bugs attributed, bug rate, and flag (colour badge). Table is sortable by any column. Default sort: flag severity.	Must have	Flag badge colours: green, yellow, red, gray. Tooltip on flag
5	Period type + period selector	As an engineering manager, I want to switch between weekly and monthly views and pick any past period so that I can review data at the cadence that matters.	Two tabs or a toggle: Weekly / Monthly. Each mode shows a period dropdown listing all periods for which data has been uploaded. Selecting a period reloads the leaderboard. Default: current period in monthly mode.	Must have	Weekly period format: 2026-W23. Monthly: 2026-05.

#	Title	User story	Acceptance criteria	Priority	Notes
6	Flag filter	As an engineering manager, I want to filter the leaderboard to show only red and yellow developers so that I can focus on who needs attention.	A filter control above the table allows multi-select by flag value. Selecting red+yellow hides green and gray rows. Default: all flags shown.	Must have	badge explains the scoring rule.
7	Team summary cards	As an engineering manager, I want to see aggregate team numbers at the top of the dashboard so that I can assess overall team health before drilling into individuals.	Above the leaderboard, show 4 summary cards: total tokens spent this period, total deliverables closed, average efficiency score, and flag distribution (count of green/yellow/red/gray).	Must have	Cards update when period is changed.

5.3 Developer profile - individual view

#	Title	User story	Acceptance criteria	Priority	Notes
8	Developer profile page	As a developer, I want to see my own metrics on a dedicated page so that I understand exactly how I am performing and what the numbers mean.	Clicking a developer name in the leaderboard opens their profile. Page shows: current period metric cards (tokens, lines, deliverables, story points, efficiency score, bug rate, flag), and a plain-language explanation of what their flag status means and why.	Must have	Plain-language flag explanation is critical - the formula is opaque without it.
9	Week-overweek trend chart	As a developer, I want to see how my metrics	A line chart on the profile page shows tokens spent,	Must have	Minimum 2 weeks of data

#	Title	User story	Acceptance criteria	Priority	Notes
		have changed week over week so that I can track short-term progress.	deliverables closed, and AI efficiency score across the last 8 weeks. X-axis: week label. Hover shows exact values. Only weeks with data are plotted.		to render the chart.
10	Month-overmonth trend chart	As a developer, I want to see how my metrics have changed month over month so that I can understand longer-term patterns.	A second line chart shows the same metrics across the last 6 months. Toggle between weekly and monthly chart on the profile page.	Must have	Minimum 2 months of data to render.
1 1	Bug attribution detail	As a developer, I want to see which bugs were attributed to me and why so that I can understand my quality score.	A section on the profile page lists each attributed bug: issue key, epic, sprint, bug created date, and the story that triggered attribution. If a Jira base URL is configured, issue keys link to Jira.	Nice to have	Requires Jira base URL config. Low-effort if Jira URL is stable.

5.4 Team trend view

#	Title	User story	Acceptance criteria	Priority	Notes
12	Team aggregate trend chart	As an engineering manager, I want to see how the team's aggregate AI usage and output are trending over time so that I can report on AI adoption to leadership.	A Trends page shows team-level charts: total tokens spent, average efficiency score, total deliverables closed - plotted across all stored periods. Period range is selectable. Toggle between weekly and monthly granularity.	Must have	Aggregate = sum or average across all developers per period.
13	Flag distribution	As an engineering	A stacked bar chart shows count of	Nice to have	Useful for QBR slides. Defer if

#	Title	User story	Acceptance criteria	Priority	Notes
	over time	manager, I want to see how the team's flag distribution has changed over time so that I can track whether quality and efficiency are improving.	green/yellow/red/gray flags per period. Hovering shows the developer names in each band.		complexity is high.
14	AI adoption rate	As an engineering manager, I want to know what percentage of the team is actively using Claude Code each period so that I can identify who has not adopted the tool yet.	A metric card on the Trends page shows: 'X of Y developers used Claude Code this period', where 'used' means tokens > 0. A secondary view lists the developers with zero token spend.	Must have	Zero-token developers must still appear in the leaderboard, not be silently excluded.

6. UX Mockups

Mockups and wireframes to be added here. The following screens are required from Design before engineering kickoff:

- Dashboard - team summary cards + leaderboard table with period selector, flag filter, and weekly/monthly toggle.
- Data upload screen - two-file upload with period picker, upload log, and error state.
- Developer profile page - metric cards, flag explanation, weekly trend chart, monthly trend chart, bug attribution list.
- Team trends page - aggregate line charts, flag distribution stacked bar, AI adoption rate card.

Work with the Design Lead to produce wireframes in Figma. Link the main Figma file here once available.

7. Questions

Question	Outcome / Owner
What exact columns does the Claude Console CSV export contain? In particular, what is the column name for tokens vs. spend (USD), and is it available at daily, weekly, or only monthly granularity?	Must be confirmed with whoever manages Claude Console before sprint 1. This determines whether weekly view is possible from the raw export or requires aggregation. Owner: Engineering Lead.
The current CLI uses 'spend_usd' and 'lines_of_code' from Claude Console. The real dashboard brief asks for 'tokens'. Are tokens and spend two separate columns in the export, or is spend the only available proxy?	If only spend is available, the dashboard labels it as 'Claude spend ($)' rather than 'tokens'. Confirm with Claude Console export sample. Owner: Navneet Shukla.
What Jira issue types count as a 'deliverable'? Today the CLI counts Stories and Tasks. Are sub-tasks, epics, or custom types also in scope?	Decision needed from Product + Engineering before build. Recommendation: Story and Task only for v1. Owner: Navneet Shukla.
For the weekly view, the Claude Console CSV must support weekly granularity. If it only exports monthly, can it be filtered by a date range to approximate a week?	Needs to be tested against an actual Claude Console export. If weekly export is not possible, the weekly view in the dashboard is limited to Jira data only (deliverables per week) with monthly Claude data. Owner: Engineering Lead.
What backend framework should be used? Options: FastAPI (Python, minimal context-switch from the existing pipeline), Django, or Node/Express (requires a bridge to the Python pipeline).	Recommendation: FastAPI. Keeps the full stack in Python. Decide with Engineering Lead before sprint 1.
What database should store historical output per period? Options: SQLite (zero infrastructure, fine for a small team), PostgreSQL (more robust at scale).	Recommendation: SQLite for v1. Upgrade path to PostgreSQL if the team grows. Owner: Engineering Lead.
What is the hosting target for the web app internal server, cloud VM (e.g. AWS EC2), or a managed PaaS?	To be decided with Engineering Lead before infrastructure setup. Affects deployment and CSV upload storage.
Should the AI efficiency score formula be explained inline in the UI for developers? (Formula: story points delivered / spend USD. Suppressed when story points coverage < 80%.)	Strong recommendation: yes. A tooltip or info card on both the leaderboard and profile page. Without explanation the score appears arbitrary. Owner: Design Lead.

8. Out of Scope

The following are explicitly not part of v1. A separate scope decision is required to include any of them:

- Authentication and login. The v1 app runs on an internal network and is treated as a trusted tool. SSO, OAuth, and individual logins are a future scope item.
- Role-based access control. All users see all developer data in v1. Restricting developers to their own data only is future scope.
- Direct API integration with Claude Console. v1 relies on manual CSV upload. Automated ingestion via the Claude API is a future phase.

- Direct API integration with Jira. v1 relies on manual Jira CSV export. Automated ingestion via the Jira REST API is a future phase.
- Email or Slack alerting when a developer's flag turns red. The dashboard is pull-based in v1.
- Scheduled / automatic pipeline runs. Data is refreshed only on manual upload in v1. Cron-based automation is future scope.
- Rewriting the existing Python CLI pipeline. The web app wraps merge.py; it does not replace it.
- Mobile-optimised layout. This is a desktop-first internal tool.
- Multi-organisation or multi-team support. The app serves the single Company_Name engineering team in v1.
- Comparative benchmarking against external industry data.